ADR 0001
Botão Excluir

Data	Status	Integrantes
05/02/2026	Aceito	Alexander Nogueira Caio Nunes de Abreu Cassiano Nunes de Abreu Gabriel Naoki Uto Turigoe Wyllian Mariano Nogueira

Sumário

Contexto 2
Decisão 2
Consequências 2
- Positivas 2
- Negativas / Riscos 3

Mitigações 3
Alternativas consideradas 3
Follow Up / Próximos passos 3
Referencias 3

Contexto

A aplicação apresentava a necessidade de definir como os contatos seriam removidos da agenda. A exclusão física direta (deletar a linha do arquivo `.txt`) trazia riscos de perda permanente de dados em caso de erros operacionais e impossibilitava auditorias futuras sobre o que foi removido.

Decisão

- Adotar a estratégia de **Soft Delete** (Exclusão Lógica). Em vez de remover o registro fisicamente, o sistema passará a utilizar um atributo de controle (ex: `ativo` ou `status`). O método de exclusão apenas alterará esse estado, e as listagens da interface gráfica deverão filtrar os dados para exibir apenas os contatos que ainda estão ativos.

```
// Exemplo da lógica de exclusão lógica no Repository/Service
public void excluirLogico(int id) {
    Contato contato = repository.findById(id);
    contato.setAtivo(false); // Marca como inativo em vez de dar 'delete'
    repository.update(contato);
}
```

Consequências

Positivas

- Aumento imediato da testabilidade (controllers testáveis sem UI).
- Redução do acoplamento entre UI e lógica; maior coesão das responsabilidades relacionadas a contatos.
- Facilita futuras extrações e refatorações (Repository/Service) por possuir pontos de encapsulamento bem definidos.
- Garante que dados de contatos não sejam perdidos permanentemente por erro de clique.

Negativas / Riscos

- Risco de acumular lógica de persistência no Controller (antipattern) se a extração não ocorrer depois.
- Possibilidade de concentração de responsabilidades no Controller (violação temporária do princípio de camada).
- Persistência em texto plano tem limitações (consistência, concorrência, segurança) — deve ser vista como solução didática e temporária.

Mitigações

- Inserir TODO e issue rastreável "Extrair Repository/Service" com plano e prazo (ex.: até N2).
- Injetar Path para permitir testes automáticos com arquivos temporários.
- Manter métodos do Controller pequenos e bem nomeados (e.g., readAllLines(), appendLine(), updateLine()), para facilitar substituição posterior.
- Usar SwingWorker para todas as operações de I/O invocadas pela UI.
- Adicionar JOptionPane.showConfirmDialog antes de chamar o método excluir para confirmar a intenção do usuário.

Alternativas consideradas

- Manter lógica e persistência nas Views (Rejeitado)
  - Simples inicialmente, porém mantém baixa testabilidade e alto acoplamento; não atende objetivo pedagógico.
- Extrair as camadas Service e Repository (Considerado, aceito para momento posterior)

Follow Up / Próximos passos

- Criar issue tracking (Se estivéssemos utilizando uma ferramenta para a gestão do projeto):
  - #Extrair-Repository-Post-N1 com checklist de tarefas (definir interface Repository, mover I/O do Controller, atualizar testes e ADR).
- Atualizar DAS e diagramas (C3/C2) para refletir Controller-first e, quando a extração ocorrer, atualizar ADR para registrar migração.
- Exigir entregas: controllers com Path injetável, testes unitários do Controller (mín. 3), Views delegando à interface e uso de SwingWorker para I/O.

Referências

- Projeto “Agenda de Contatos” -> repository: https://github.com/marquesclayton/ads3.contatos-camadas
- DAS — sessões “Atualização: Estratégia incremental de evolução (Controllerfirst)”
- Fowler — Strangler Fig Pattern (para migração incremental)